

AI Agent Development Frameworks Comparison

Quick-Reference Matrix (Page 1)

Criterion / Feature	LangChain	LangGraph	CrewAI	AutoGPT	BabyAGI	AgentScope
Explicitly models a state machine	✗	✓	✗	✗	✗	✗
Framework-agnostic (can swap LLMs, tools, memory)	✓	✓	✓	✓	✓	✓
Asynchronous event-based orchestration	✗	✓ (via nodes/edges)	✗	✗	✗	✓ (event loop)
Built for core web-service logic (REST/gRPC ready)	✓ (via chains/tools)	✓ (via graph API)	✓ (crew as service)	✓ (agent as API)	✓ (simple API)	✓ (gateway)
Open-source UI for monitoring / tracing	✓ (LangSmith)	✗	✗	✗	✗	✓ (built-in dashboard)
Works with non-LLM use-cases (pure logic, data pipelines)	✓ (chains can be pure)	✓ (graph can be pure)	✓ (crew can run scripts)	✗ (LLM-centric)	✗ (LLM-centric)	✓ (can call any tool)
Supports multi-agent collaboration (role-based, handoff)	✗ (single-agent chains)	✗ (single-agent graph)	✓ (crew with roles)	✓ (multiple agents)	✗ (single-agent loop)	✓ (agent-scope hierarchy)
Built-in memory / state persistence	✓ (via memory modules)	✓ (graph state)	✓ (shared crew memory)	✗ (short-term)	✗ (short-term)	✓ (agent memory)
Rich tool / plugin ecosystem	✓ (hundreds of tools)	✓ (can call any tool)	✓ (crew tools)	✓ (tool use)	✓ (basic tools)	✓ (plug-in framework)
Human-in-the-loop / approval steps	✓ (interrupt, callbacks)	✓ (node interrupts)	✓ (human review tasks)	✗	✗	✓ (approval gates)
Code-first / low-code UI	✓ (Python/TS)	✓ (Python)	✓ (Python/YAML)	✓ (Python)	✓ (Python)	✓ (Python/TS)
Production-grade scaling / reliability	✓ (via LangServe)	✓ (LangGraph Server)	✗ (mostly dev)	✗ (prototype)	✗ (prototype)	✓ (gateway + workers)

Quick-Reference Matrix (Page 2)

Criterion / Feature	SuperAgent	Apache Burr	Apache Hamilton	Temporal	Semantic Kernel	LlamaIndex*
Explicitly models a state machine	✗	✓	✗	✗	✗	✗
Framework-agnostic (can swap LLMs, tools, memory)	✓	✓	✓	✓	✓	✓
Asynchronous event-based orchestration	✓	✗	✗	✓ (workflow engine)	✗	✗
Built for core web-service logic (REST/gRPC ready)	✓	✓ (state-machine service)	✓ (dataflow service)	✓ (workflow as a service)	✓ (SDK for services)	✓ (indexing service)
Open-source UI for monitoring / tracing	✓ (web UI)	✓ (Burr UI)	✓ (Hamilton UI)	✗ (Temporal UI separate)	✓ (Azure AI Studio)	✗
Works with non-LLM use-cases (pure logic, data pipelines)	✓	✓ (state machine)	✓ (dataflow)	✓ (workflow engine)	✓ (can call any function)	✓ (indexing without LLM)
Supports multi-agent collaboration (role-based, handoff)	✓ (agent teams)	✗	✗	✗	✗	✗
Built-in memory / state persistence	✓	✓ (state persistence)	✓ (step-level state)	✓ (workflow state)	✓ (memory connectors)	✓ (vector store)
Rich tool / plugin ecosystem	✓	✓ (hooks)	✓ (plug-ins)	✓ (activities)	✓ (skills)	✓ (loaders, wrappers)
Human-in-the-loop / approval steps	✓	✓ (interrupts)	✓ (step approval)	✓ (human tasks)	✓ (approvals)	✗
Code-first / low-code UI	✓	✓ (YAML/JSON)	✓ (Python)	✓ (Java/Go SDK)	✓ (C#/Python/Java)	✓ (Python)
Production-grade scaling / reliability	✓	✓ (state-machine runtime)	✓ (dataflow runtime)	✓ (temporal service)	✓ (Azure integration)	✗ (mostly dev)

Quick-Reference Matrix (Page 3)

Criterion / Feature	Guidance	PromptFlow	Haystack	AgentGPT	MetaGPT	Autogen (Microsoft)
Explicitly models a state machine	✗	✗	✗	✗	✗	✗
Framework-agnostic (can swap LLMs, tools, memory)	✓	✓	✓	✓	✓	✓
Asynchronous event-based orchestration	✗	✓ (async steps)	✗	✗	✗	✓ (event-driven)
Built for core web-service logic (REST/gRPC ready)	✓ (templating service)	✓ (orchestration service)	✓ (search-as-service)	✓ (agent API)	✓ (multi-agent API)	✓ (REST API)
Open-source UI for monitoring / tracing	✗	✗	✗	✗	✗	✓ (Dashboard)
Works with non-LLM use-cases (pure logic, data pipelines)	✓ (templating)	✓ (steps can be code)	✓ (retrieval pipelines)	✗	✗	✓ (pure logic)
Supports multi-agent collaboration (role-based, handoff)	✗	✗	✗	✓ (multiple agents)	✓ (agent-society simulation)	✓ (multi-agent)
Built-in memory / state persistence	✗	✗	✓ (document store)	✗	✗	✓ (vector/ KV)
Rich tool / plugin ecosystem	✓ (templates)	✓ (components)	✓ (integrations)	✓ (tool use)	✓ (agent tools)	✓ (tools)
Human-in-the-loop / approval steps	✓ (prompt editing)	✓ (approval steps)	✗	✗	✗	✗
Code-first / low-code UI	✓ (DSL)	✓ (YAML/JSON)	✓ (Python)	✓ (Python)	✓ (Python)	✓ (Python)
Production-grade scaling / reliability	✗	✓ (managed service)	✓ (Haystack pipelines)	✗	✗	✓ (Azure/ Docker)

Quick-Reference Matrix (Page 4)

Criterion / Feature	PydanticAI	Langroid	Letta (MemGPT)	Agno (Phidata)	smolagents	AutoGen / AG2
Explicitly models a state machine	✗	✗	✗	✓	✗	✗
Framework-agnostic (can swap LLMs, tools, memory)	✓	✓	✓	✓	✓	✓
Asynchronous event-based orchestration	✗	✓ (async)	✓ (async)	✓ (async)	✓ (async)	✓ (async)
Built for core web-service logic (REST/gRPC ready)	✓ (FastAPI)	✓ (REST/gRPC)	✓ (API)	✓ (API)	✓ (API)	✓ (API)
Open-source UI for monitoring / tracing	✗	✗	✗	✗	✗	✗
Works with non-LLM use-cases (pure logic, data pipelines)	✓ (pure Python)	✓ (tool-agnostic)	✓ (state)	✓ (functional)	✓ (functional)	✓ (functional)
Supports multi-agent collaboration (role-based, handoff)	✓ (role-based)	✓ (role-based)	✓ (role-based)	✓ (role-based)	✓ (role-based)	✓ (multi-agent)
Built-in memory / state persistence	✓ (pydantic models)	✓ (memory)	✓ (memGPT memory)	✓ (state)	✓ (state)	✓ (state)
Rich tool / plugin ecosystem	✓ (integrations)	✓ (tools)	✓ (tools)	✓ (tools)	✓ (tools)	✓ (tools)
Human-in-the-loop / approval steps	✓ (approval)	✓ (approval)	✓ (approval)	✓ (approval)	✓ (approval)	✓ (approval)
Code-first / low-code UI	✓ (Python)	✓ (Python)	✓ (Python)	✓ (Python)	✓ (Python)	✓ (Python)
Production-grade scaling / reliability	✓ (Pydantic-based)	✓ (scalable)	✓ (MemGPT server)	✓ (deployable)	✓ (lightweight)	✓ (deployable)

* *LlamaIndex* is primarily a data-indexing / retrieval framework; it can be used as the “memory” layer for agents but does not itself orchestrate agent behavior.

Detailed Comparison Table

Framework	Advantages	Disadvantages	Typical Uses	Core Capabilities
-----------	------------	---------------	--------------	-------------------

LangChain	• Mature, extensive library of chains, agents, tools • LLM-agnostic (OpenAI, Anthropic, HuggingFace, local) • Strong ecosystem (LangServe, LangSmith) • Easy to prototype RAG, tool-use, chat-bots	• Verbose for complex flows • No built-in state-machine visualisation • Orchestration is linear unless you add custom logic	• Retrieval-augmented generation • Tool-enabled chat agents • Simple autonomous agents • Prototyping LLM pipelines	• Chains (LLM → prompt → output) • Agents (zero-shot, ReAct, etc.) • Memory modules • Tool callbacks • Vector-store integrations
LangGraph	• Explicit graph-based state machine (nodes=steps, edges=transitions) • Supports conditional branching, loops, parallelism • Persistent checkpointing (via SQLite/Postgres) • Integrates with LangChain tools	• Newer, smaller community • Requires learning graph concepts • UI for tracing less mature than LangSmith	• Complex multi-step agents with branching • Workflow-like LLM pipelines (e.g., research → synthesis → critique) • Human-in-the-loop approvals	• Nodes (any callable: LLM, tool, function) • Edges (conditional, deterministic) • State persistence • Streaming & async execution • Interrupt / resume
CrewAI	• Role-based multi-agent out-of-the-box • Simple YAML/JSON crew definition • Built-in task delegation & handoff • Good for simulations & collaborative problem-solving	• Primarily Python-only • Limited tooling compared to LangChain • Less focus on single-agent deep reasoning	• Multi-agent research teams • Automated content creation (writer + reviewer + editor) • Education / training simulations	• Agents with defined role, goal, backstory • Task queue & handoff mechanism • Shared memory (optional) • Process orchestration (sequential / hierarchical)
AutoGPT	• One of the first fully autonomous agents (goal → self-prompting) • Demonstrates emergent self-improvement loops • Easy to run locally with Docker	• Often unstable; can loop or get stuck • Limited controllability (hard to steer) • Sparse documentation & tooling	• Demonstrations of autonomous goal-seeking • Prototype self-directed research • Exploratory AI safety experiments	• Goal-driven prompting loop • Self-generated subtasks • Tool use (web, file, code) • Short-term memory (vector store)
BabyAGI	• Minimalist task-generation loop (goal → task creation → execution → result storage) • Easy to understand & extend • Works with any LLM via API	• Very basic; no built-in tool abstraction • No persistence beyond simple storage • Not suited for complex workflows	• Teaching autonomous task management • Prototyping simple goal-driven agents • Basis for custom agent loops	• Task generation (LLM proposes next task) • Execution (LLM or tool) • Result storage (vector DB) • Repetition until goal met

AgentScope	• Hierarchical agent scopes (parent/child) • Built-in message passing, event loop • Supports synchronous & async modes • Includes dashboard for monitoring	• Smaller ecosystem; fewer ready-made tools • Learning curve for scope hierarchy	• Distributed agent systems • Multi-robot or IoT agent coordination • Research on agent hierarchies	• Agent tree with scoped contexts • Async message bus • LLM & tool invocation hooks • Persistence & checkpointing
SuperAgent	• Focus on reusable agent “skills” (plug-ins) • Visual skill composition UI • Supports both synchronous & async execution • Good for building agent marketplaces	• Still early-stage; limited third-party skills • UI may be overkill for simple scripts	• Marketplace of agent capabilities • Rapid assembly of agents from skill blocks • Enterprise agent orchestration	• Skill registry (LLM prompt, tool use, memory) • Skill chaining & nesting • Runtime monitoring UI • Event-driven triggers
Apache Burr	• Explicit state-machine model (states, transitions, actions) • Built-in UI for visualising & debugging state machines • Language-agnostic (Java, Kotlin, etc.) • Works for non-LLM workflows as well	• Primarily JVM-based (less native Python support) • Heavier weight than pure-Python libs • Requires defining states upfront	• Workflow automation with LLM steps • Legacy system integration where state matters • Any process that benefits from state-machine clarity	• States & transitions • Actions (can call LLM, tools, services) • Persistence & snapshotting • UI designer & simulator
Apache Hamilton	• Data-centric micro-functional programming (functions → dataflow) • Strong typing & automatic DAG generation • Easy to unit-test each function • Good for data pipelines that occasionally call LLMs	• Not designed for complex control flow (loops, conditionals require extra wrappers) • Learning curve for function-only style	• ETL / data enrichment pipelines with LLM steps • Feature engineering where LLMs generate labels • Reproducible ML workflows	• Pure Python functions as nodes • Automatic DAG construction from function signatures • Caching & validation • Pluggable data loaders/savers

Temporal	• Battle-tested workflow engine (durable, scalable, fault-tolerant) • Supports long-running async workflows (years) • Built-in retry, timeout, versioning • Language SDKs (Java, Go, PHP, Python, TypeScript)	• Overkill for simple agents • Requires running a Temporal cluster • Workflow code must be deterministic (no nondeterministic LLM calls without activities)	• Long-running autonomous agents (e.g., continuous monitoring) • Multi-step business processes with LLM checks • Stateful agent services needing durability	• Workflows (sequential/parallel) • Activities (any task: LLM call, tool, service) • Timers, signals, child workflows • Durable execution & visibility
Semantic Kernel	• Microsoft-backed, enterprise-grade • Strong integration with Azure AI services • Planner component for autonomous goal-driven execution • Supports skills, memories, connectors	• Tied to Microsoft ecosystem (though open-source) • Slightly heavier than minimalist libs • Documentation can be dense	• Enterprise copilots & assistants • Azure-hosted agent solutions • Scenarios needing built-in planner & memory	• Planner (goal → plan → execution) • Skills (LLM prompts, native functions) • Memory (volatile & long-term) • Connectors (Azure Services, external APIs)
LlamaIndex	• Best-in-class data ingestion & indexing for LLMs • Variety of loaders, vector stores, node parsers • Simple query interface (retrieval-augmented generation)	• Not an orchestration framework; you still need a chain/agent to act on retrieved data • Some advanced features require paid LlamaCloud	• Retrieval-augmented generation pipelines • Knowledge-base chatbots • Document-centric LLM applications	• Data connectors (files, databases, APIs) • Indexing (vector, keyword, graph) • Query engines (retrieval, summarisation, chat)
Guidance	• Enables precise control over LLM generation (constrained decoding, templating) • Works with many backends (HuggingFace, LlamaCPP, OpenAI) • Allows mixing generation with traditional code	• Focuses on generation control, not workflow orchestration • Requires learning the templating language • Less suited for multi-agent scenarios	• Prompt engineering with guaranteed output format • Structured data extraction from LLMs • Grammar-guided generation (JSON, SQL, etc.)	• Templating language with loops, conditionals • Token-level constraints (regex, grammar) • Integration with external Python code

PromptFlow (Azure)	• Visual DAG authoring for LLM flows • Built-in testing, debugging, CI/CD integration • Easy deployment as Azure ML endpoint • Supports branching, loops, parallelism	• Azure-centric (though open-source core) • Requires Azure account for full managed features • Learning curve for the UI	• Production LLM services (copilots, chatbots) • Experimental point engineering • Batch LLM processing jobs	• Flow definition (nodes = LLMs, tools, code) • Conditional branches & loops • Testing & debugging tools • Deployment as REST endpoint
Haystack	• Strong focus on retrieval-augmented generation & question answering • Rich set of document stores (Elasticsearch, FAISS, Milvus, etc.) • Pipelines are easy to compose & extend • Good for search-centric LLM apps	• Less general-purpose for non-search agent logic • Pipeline DSL can be verbose for simple tasks • Primarily Python/Java	• Enterprise QA / chat-bot over internal docs • Semantic search + LLM generation • Retrieval-heavy agent workflows	• Nodes (retriever, ranker, reader, generator) • Document stores (SQL, ES, vector) • Pre- & post-processing components • Evaluation & benchmarking tools
AgentGPT	• Web-based UI for creating & running autonomous agents • Goal-driven looping similar to AutoGPT but with UI controls • Ability to set tools & limits via UI	• Limited to what the UI exposes; harder to extend via code • Still prone to looping / getting stuck • Primarily a demo / prototyping tool	• Quick prototyping of autonomous agents • Demonstrations for non-technical stakeholders • Educational tool for agent concepts	• Goal input → LLM-driven task loop • Tool use (web, file, API) via UI config • Iteration & termination controls
MetaGPT	• Simulates a software company (product manager, architect, engineer, QA) • Generates entire codebases from a single requirement • Strong focus on software engineering tasks • Open-source with active community	• Narrow domain (software development) • Generated code may need review/tuning • Requires understanding of the “company” roles	• Automated software development from specs • Code generation & refactoring assistants • Teaching SDLC with AI agents	• Role-based agents (PM, Architect, Engineer, QA) • Standard Operating Procedures (SOPs) for each role • Artifact exchange (docs, diagrams, code) • Self-reflection & critique loops
Autogen (Microsoft)	• Multi-agent conversation framework • Strong support for LLMs with tool use • Built-in caching and skill sharing • Easy to define agent groups and workflows	• Still evolving; documentation scattered • Requires learning agent-chat patterns	• Conversational agents • Collaborative problem-solving with multiple LLMs • Auto-generated code & documentation	• Agent chat (LLM ↔ LLM) • Tool usage & registration • Skill sharing & caching • Group chat and nested conversations

PydanticAI	• Type-safe agent building with Pydantic models • Declarative definition of agent behavior • Generates structured outputs (JSON) automatically • Integrates with FastAPI for serving	• Smaller community compared to LangChain • Focus on typed contracts may feel restrictive	• Structured data extraction agents • API-backed agents with validation • Workflows needing guaranteed output schemas	• Pydantic-based agent definition • Automatic validation & serialization • Tool integration via dependencies • FastAPI endpoint generation
Langroid	• Lightweight, modular agent library • Supports multiple LLMs and embedding models • Built-in task queues and agent orchestration • Easy to extend with custom tools	• Fewer ready-made tools than LangChain • Documentation still growing	• Simple conversational agents • Retrieval-augmented QA pipelines • Agent-based data processing	• Agent base class with LLM & tool hooks • Task queue for asynchronous work • Memory modules (vector/keyword) • Optional chat UI
Letta (MemGPT)	• Memory-augmented agents with unlimited context via paging • Combines LLMs with external memory (vector/store) • Designed for long-running conversations • Open-source with active community	• Requires managing memory backend • Can be complex to tune memory parameters • Less focus on multi-agent collaboration	• Long-context chatbots • Document-based research assistants • Agents needing persistent memory across sessions	• Memory hierarchy (RAM ↔ disk) • LLM core with memory functions • Token-paging mechanism • Tool use API
Agno (Phidata)	• Framework for building autonomous agents with data integration • Strong focus on data-driven agents (SQL, NoSQL, APIs) • Provides agent services (REST, gRPC) out-of-the-box • Includes monitoring and logging	• More oriented toward data engineers than pure AI researchers • Requires setting up data sources	• Data-analysis agents • Automated ETL with LLM reasoning • Internal tooling agents for ops	• Agent base with data connectors • Service wrapper (REST/gRPC) • Built-in scheduler & cron • Monitoring dashboards

smolagents	- Minimalist agent library aiming for simplicity - Very small codebase, easy to read and modify - Supports LLMs via simple API - Good for teaching and prototyping	- Limited features; no advanced memory or tooling - Not suited for production-scale systems	- Educational prototypes - Quick proof-of-concept agents - Experiments with LLM loops	- Core agent loop (LLM → action → observation) - Simple tool interface - Optional memory slab
AutoGen / AG2	- Evolution of AutoGen with improved stability - Multi-agent conversation with configurable agent types - Supports LLM caching, tool use, and skill sharing - Actively maintained by Microsoft and community	- Still relatively new; breaking changes possible - Requires learning the agent-conversation paradigm	- Conversational AI systems - Collaborative code generation - Agent-based simulation and gaming	- Agent conversation framework - LLM caching & retry policies - Tool registration & execution - Skill library for reusable behaviors

How to Use This Information

- Pick a framework** based on the primary dimension you need:
 - State-machine & visual debugging* → **Apache Burr** or **LangGraph**.
 - Multi-agent role play* → **CrewAI**, **MetaGPT**, **AgentScope**, **Autogen**, **Langroid**.
 - Durable, long-running services* → **Temporal** (or **SuperAgent** with persistence).
 - Simple prototyping & huge tool ecosystem* → **LangChain**.
 - Enterprise-grade with Microsoft Azure* → **Semantic Kernel** or **PromptFlow**.
 - Retrieval-heavy QA* → **LlamaIndex** + **Haystack** (often paired with LangChain/LangGraph).
 - Type-safe, structured outputs* → **PydanticAI**.
 - Unlimited context / memory* → **Letta (MemGPT)**.
 - Data-centric agents* → **Agno (Phidata)**.
 - Minimalist, teaching-focused* → **smolagents**.
 - Conversational multi-LLM systems* → **AutoGen / AG2**.
- Combine** where helpful: many teams use LlamaIndex for retrieval + LangGraph for orchestration + Guidance for strict output formatting; or combine Letta for memory with any orchestrator.
- Check the matrix** for any “must-have” feature (e.g., human-in-the-loop, async event-based, non-LLM usage) before committing.

Feel free to ask for deeper dives into any specific framework, sample code snippets, or guidance on integrating multiple tools!